Serialize/Deserialize in Dart

- Data Type “dynamic” in Dart
- Maps (also called Dictionaries)
- Serialize and Deserialize to a json file
- First look at asynchronous Methods

Data type “dynamic” – compare Python and Dart

dynamic.py > ...

```python

d = "abc"
d = 1
d = d + 1
print (f"d is {d}")

d = [4,5,6]
print(type(d))
if (type(d) is list):
    print (f"d[1] is {d[1]}")
else:
    print("d is no list")

```

```
PS C:\flutter\code\python_samples> & C:/flutter/code/python_samples/.venv/Scripts/Activate.ps1
(.venv) PS C:\flutter\code\python_samples>  & 'c:\flutter\code\python_samples\.venv\Scripts\python.exe' 'c:\Users\GS\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '53155' '--' 'C:\flutter\code\python_samples\dynamic.py'
d is 2
<class 'list'>
d[1] is 5
```

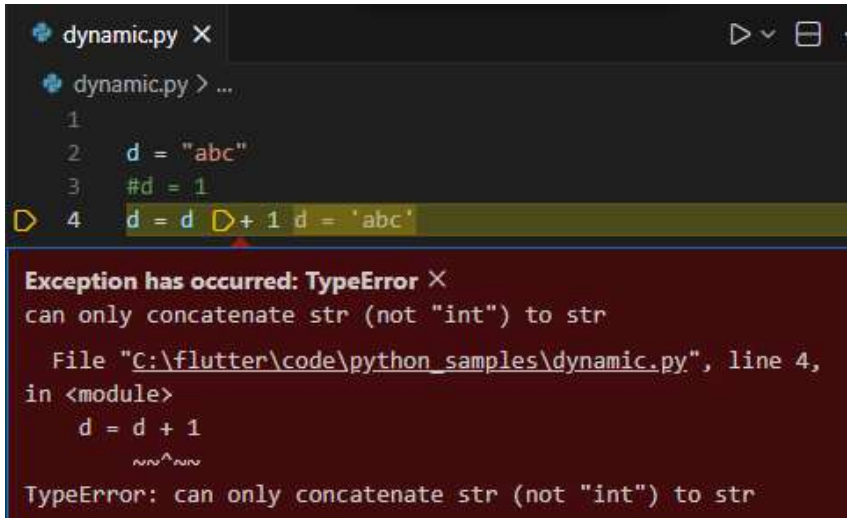
dynamic.dart > ...

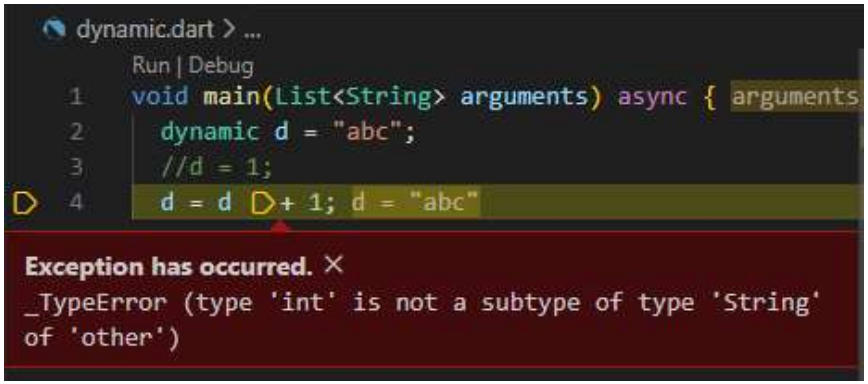
Run | Debug

```dart
void main(List<String> arguments) async {
  dynamic d = "abc";
  d = 1;
  d = d + 1;
  print("d is $d");

  d = [4, 5, 6];
  print(d.runtimeType);
  if (d is List<int>) {
    print("d[1] is ${d[1]}");
  } else {
    print("d is no list");
  }
}
```

```
Connecting to VM Service at ws://127.0.0.1:56809/0BcktIgcQEc=/ws
Connected to the VM Service.
d is 2
List<int>
d[1] is 5

Exited.
```

Errors occur at runtime, not at compile-time





dynamic vs. Object

```dart
void main(List<String> arguments) async {
  Object d = "abc";
  d = 1;
  d = d + 1;
  print("d is $d");

  d = [4, 5, 6];
  print("d.length is ${d.length}");
  print(d.runtimeType);
  if (d is List<int>) {
    print("d[1] is ${d[1]}");
  } else {
    print("d is no list");
  }
}
```

All classes in Dart are derived from Object.

Therefore instances of every class can be assigned to a variable of type Object.

For a variable of type Object, compiler allows to call only methods and operators of type object. Not .length or operator+.

Dictionaries in Dart

ChatGPT

Dictionaries in Dart (called Maps)

In Dart, dictionaries are called **Maps**. A `Map` stores data as **key–value pairs**, where each key is unique.

Creating a Map

dart Copy code

```dart
Map<String, int> ages = {
  'Alice': 30,
  'Bob': 25,
};
```

You can also use type inference:

dart Copy code

```dart
var scores = {'Math': 90, 'English': 85};
```

Accessing & Updating Values

dart

```dart
print(ages['Alice']); // 30

ages['Charlie'] = 28; // add
ages['Bob'] = 26;     // update
```

Common Operations

dart

```dart
ages.remove('Alice');
ages.containsKey('Bob');    // true
ages.containsValue(26);     // true
ages.length;                // number of entries
```

Null Safety Note

Accessing a key that doesn't exist returns `null`, so watch out with non-nullable types.

Map example in Dart

```dart
import 'employee.dart';

Run | Debug
void main() {
  var fritz = Employee("1234");
  fritz.name = "Fritz Fischer";
  fritz.hoursPerWeek = 35;
  fritz.salary = 3500;

  var karin = Employee("2323");
  karin.name = "Karin Schmitt";
  karin.hoursPerWeek = 20;
  karin.salary = 2500;

  Map<String, Employee> employees = {};
  employees["1234"] = fritz;
  employees[karin.id] = karin;

  for (var entry in employees.entries) {
    print("${entry.key} ${entry.value.name}");
  }
  employees.remove("2323");
  print(employees.length);
  for (var key in employees.keys) {
    print("$key ${employees[key]!.name}");
  }
}
```

```dart
class Employee {
  Employee(this.id);
  String id;
  String name = "unknown";
  double hoursPerWeek = 40;
  double salary = 4000;
}
```

Compare Map between Dart and Python

```dart
import 'employee.dart';

Run | Debug
void main() {
  var fritz = Employee("1234");
  fritz.name = "Fritz Fischer";
  fritz.hoursPerWeek = 35;
  fritz.salary = 3500;

  var karin = Employee("2323");
  karin.name = "Karin Schmitt";
  karin.hoursPerWeek = 20;
  karin.salary = 2500;

  Map<String, Employee> employees = {};
  employees["1234"] = fritz;
  employees[karin.id] = karin;

  for (var entry in employees.entries) {
    print("${entry.key} ${entry.value.name}");
  }
  employees.remove("2323");
  print(employees.length);
  for (var key in employees.keys) {
    print("$key ${employees[key]!.name}");
  }
}
```

```python
from employee import Employee



fritz = Employee("1234")
fritz.name = "Fritz Fischer"
fritz.hoursPerWeek = 35
fritz.salary = 3500

karin = Employee("2323")
karin.name = "Karin Schmitt"
karin.hoursPerWeek = 20
karin.salary = 2500

employees = {}
employees["1234"] = fritz
employees[karin.id] = karin

for key, value in employees.items():
    print(f"{key} {value.name}")
# next line without None: exception if key does not exist
employees.pop("2323",None)
print(len(employees))
for key in employees:
    print(f"{key} {employees[key].name}")
```

```python
class Employee:
    # The __init__ method is the constructor
    def __init__(self, id):
        self.id = id    # Instance attribute
        self.name = "unknown"
        self.hoursPerWeek = 40
        self.salary = 4000
```

Not existing key in python throws exception

```
27  id = "xyz"
28  employee = employees[id]  employees = {'1234': <employee.Employee

Exception has occurred: KeyError ×
'xyz'
  File "C:\flutter\code\dart_serialization\map.py", line 28, in <module>
    employee = employees[id]
               ~~~~~~~~~~^^^^
KeyError: 'xyz'
```

.get(id) returns None:

```
id = "xyz"
#employee = employees[id]
employee = employees.get(id)
if (employee is None):
    print(f"not matching id {id}")
```

in Dart:

```
var id = "xyz";
var employee = employees[id];
if (employee == null) {
  print("did not find employee with id=$id");
}
```

Lookup in Maps versus Lists

```
void fillAndSearchMap(int employeeCount, int loopCount) {
  Map<String, Employee> employees = {};
  print("start inserting in map ...");
  for (int i = 0; i < employeeCount; i++) {
    var id = i.toString();
    employees[id] = Employee(id);
  }

  print("start searching in map ...");
  var random = Random();
  for (int i = 0; i < loopCount; i++) {
    if (i % 1000 == 0) {
      print("i is $i");
    }
    var id = random.nextInt(employeeCount - 1).toString();
    var employee = employees[id];
    if (employee == null) {
      print("did not find employee with id=$id");
    } else if (employee.id != id) {
      print("not matching id");
    }
  }
}
```

```
void fillAndSearchList(int employeeCount, int loopCount) {
  List<Employee> employees = [];
  print("start inserting in list...");
  for (int i = 0; i < employeeCount; i++) {
    employees.add(Employee(i.toString()));
  }

  print("start searching in list ...");
  var random = Random();
  for (int i = 0; i < loopCount; i++) {
    if (i % 1000 == 0) {
      print("i is $i");
    }
    var id = random.nextInt(employeeCount - 1).toString();
    var found = false;
    for (var employee in employees) {
      if (employee.id == id) {
        found = true;
        break;
      }
    }
    if (!found) {
      print("did not find employee with id=$id");
    }
  }
}
```

Lookup in a Map is faster than in a List.

Serialization to json files

JSON stands for **JavaScript Object Notation.**

Despite having "JavaScript" in the name, it is a **language-independent** data format. This means that while it looks like JavaScript object syntax, almost every modern programming language (like Dart, Python, Java, or C++) has tools to read and write it.

```
class Employee {
  Employee(this.id);
  String id;
  String name = "unknown";
  double hoursPerWeek = 40;
  double salary = 1234;
```

```
final employee = Employee("gs123");
employee.name = "GS retired";
employee.hoursPerWeek = 10;
employee.salary = 0;
```

```
// Object -> Map
Map<String, dynamic> toJson() {
  return {
    "id": id,
    "name": name,
    "hoursPerWeek": hoursPerWeek,
    "salary": salary,
  };
}
```

```
{
  "id": "gs123",
  "name": "GS retired",
  "hoursPerWeek": 10.0,
  "salary": 0.0
}
```

```
var map = employee.toJson();
var jsonString = jsonEncode(map);
print(jsonString);
```

```
{"id":"gs123","name":"GS retired","hoursPerWeek":10.0,"salary":0.0}
```

“Pretty print”

```
var map = employee.toJson();
var jsonString = jsonEncode(map);
print(jsonString);
```

```
{"id":"gs123","name":"GS retired","hoursPerWeek":10.0,"salary":0.0}
```

```
var map = employee.toJson();
var encoder = const JsonEncoder.withIndent('  ');
var jsonString = encoder.convert(map);
print(jsonString);
```

```
{
  "id": "gs123",
  "name": "GS retired",
  "hoursPerWeek": 10.0,
  "salary": 0.0
}
```

Serialize to a json file

```dart
void main() async {
  final employee = Employee("gs123");
  employee.name = "GS retired";
  employee.hoursPerWeek = 10;
  employee.salary = 0;

  var map = employee.toJson();
  // var jsonString = jsonEncode(map);
  // print(jsonString);

  var encoder = const JsonEncoder.withIndent('  ');
  var jsonString = encoder.convert(map);
  print(jsonString);

  final file = File('test.json');
  await file.writeAsString(jsonString);
  print('Saved to file.');

  String readFromFile = await file.readAsString();
  print(readFromFile);
}
```

Content of “test.json” file:

```
{
  "id": "gs123",
  "name": "GS retired",
  "hoursPerWeek": 10.0,
  "salary": 0.0
}
```

Short info on asynchronous methods

We will have a look at them later in more detail. In short for the moment:

Asynchronous methods return a “Future” value:

```
/// Reads the entire file contents as a string using the given
/// [Encoding].
///
/// Returns a `Future<String>` that completes with the string once
/// the file contents has been read.
Future<String> readAsString({Encoding encoding = utf8});
```

For such methods you must use “await” to get the return value when the method has finished:

```
String readFromFile = await file.readAsString();
```

Methods that internally use await must be declared as “async”:

```
void main() async {
```

```
Future<int> main() async {
```

Deserialization from json files

```
String readFromFile = await file.readAsString();
print(readFromFile);
```

```
final Map<String, dynamic> readMap = jsonDecode(readFromFile);
```

```
final created = Employee.fromJson(readMap);
```

```
print(
  "created employee: id: ${created.id}, name: ${created.name}, "
  "salary: ${created.salary}, hoursPerWeek: ${created.hoursPerWeek}",
);
```

```
created employee: id: gs123, name: GS retired, salary: 0.0, hoursPerWeek: 10.0
```

```
{
  "id": "gs123",
  "name": "GS retired",
  "hoursPerWeek": 10.0,
  "salary": 0.0
}
```

```
// Map -> Object
factory Employee.fromJson(Map<String, dynamic> json) {
  if (json["id"] == null) {
    throw Exception("no field 'id' found in $json");
  }
  var result = Employee(json["id"]);
  result.name = json['name'];
  result.hoursPerWeek = json['hoursPerWeek'];
  result.salary = json['salary'];
  return result;
}
```

“Normal” versus “Factory” constructor

explain the difference between a "normal" and a factory constructor in Dart

In Dart, the difference between a **generative (normal)** constructor and a **factory** constructor comes down to one core question: *Are you definitely creating a new instance of this specific class, or are you just asking the class to provide "an" object?*

Here is the breakdown of how they differ in behavior and purpose.

---

1. The Normal (Generative) Constructor

A generative constructor is the standard way to create an object. When you call it, the Dart runtime automatically allocates memory for a **new** instance and binds `this` to that instance.

- **Key Rule:** It *always* returns a new instance of the class it belongs to.
- **The "this" keyword:** You can use `this` because the object already exists by the time the constructor body runs.

```
class Employee {
  Employee(this.id);
  String id;
  String name = "unknown";
  double hoursPerWeek = 40;
  double salary = 4000;
```

```
final employee = Employee("gs123");
```

2. The Factory Constructor

A factory constructor uses the `factory` keyword. Unlike a normal constructor, it doesn't automatically create a new instance of its own class. Instead, it **returns** an instance manually.

- **Key Rule:** It can return an existing instance (from a cache), or even an instance of a **subclass**.
- **No "this":** Because a factory might not be creating a "new" instance of the current class, you cannot use `this` inside the body.
- **Explicit Return:** You must use the `return` keyword.

```
factory Employee.fromJson(Map<String, dynamic> json) {
  if (json["id"] == null) {
    throw Exception("no field 'id' found in $json");
  }
  var result = Employee(json["id"]);
  result.name = json['name'];
  result.hoursPerWeek = json['hoursPerWeek'];
  result.salary = json['salary'];
  return result;
}
```

```
final created = Employee.fromJson(readMap);
```

Serializing objects that contain lists

```dart
class Employee {
  Employee(this.id);
  String id;
  String name = "unknown";
  double hoursPerWeek = 40;
  double salary = 4000;
  List<String> languages = [];

  // Object -> Map
  Map<String, dynamic> toJson() {
    return {
      "id": id,
      "name": name,
      "hoursPerWeek": hoursPerWeek,
      "salary": salary,
      "languages": languages,
    };
  }
```

```dart
void main() async {
  final employee = Employee("gs123");
  employee.name = "GS retired";
  employee.hoursPerWeek = 10;
  employee.salary = 0;
  employee.languages = ["C#", "dart"];

  var map = employee.toJson();
  // var jsonString = jsonEncode(map);
  // print(jsonString);

  var encoder = const JsonEncoder.withIndent('  ');
  var jsonString = encoder.convert(map);
  print(jsonString);
```

```
{
  "id": "gs123",
  "name": "GS retired",
  "hoursPerWeek": 10.0,
  "salary": 0.0,
  "languages": [
    "C#",
    "dart"
  ]
}
```

Deserializing objects that contain lists

```
// Map -> Object
factory Employee.fromJson(Map<String, dynamic> json) {
  if (json["id"] == null) {
    throw Exception("no field 'id' found in $json");
  }
  var result = Employee(json["id"]);
  result.name = json['name'];
  result.hoursPerWeek = json['hoursPerWeek'];
  result.salary = json['salary'];
  result.languages = json['languages'];
  return result;
```

```
25      var result = Employee(json["id"]);
26      result.name = json['name'];
27      result.hoursPerWeek = json['hoursPerWeek'];
28      result.salary = json['salary'];
29      result.languages = json['languages'];
```

Exception has occurred. ×
_TypeError (type 'List<dynamic>' is not a subtype of type 'List<String>')

```
30      return result;
31    }
32  }
```

```
result.salary = json['salary'];
//result.languages = json['languages'];
//print(json['languages'].runtimeType); // List<dynamic>
result.languages = List<String>.from(json['languages']);
```

```
(new) List<String> List.from(Iterable<dynamic> elements, {bool growable = true})
```

Serializing & Deserializing objects that contain lists

```dart
void main() async {
  final employee = Employee("gs123");
  employee.name = "GS retired";
  employee.hoursPerWeek = 10;
  employee.salary = 0;
  employee.languages = ["C#", "dart"];

  var map = employee.toJson();
  // var jsonString = jsonEncode(map);
  // print(jsonString);

  var encoder = const JsonEncoder.withIndent('  ');
  var jsonString = encoder.convert(map);
  print(jsonString);

  final file = File('test.json');
  await file.writeAsString(jsonString);
  print('Saved to file.');

  String readFromFile = await file.readAsString();
  print(readFromFile);

  final Map<String, dynamic> readMap = jsonDecode(readFromFile);
  final created = Employee.fromJson(readMap);

  print(
    "created employee: id: ${created.id}, name: ${created.name},\n"
    "salary: ${created.salary}, hoursPerWeek: ${created.hoursPerWeek}\n"
    "languages: ${created.languages}",
  );
```

"test.json" file:

```json
{
  "id": "gs123",
  "name": "GS retired",
  "hoursPerWeek": 10.0,
  "salary": 0.0,
  "languages": [
    "C#",
    "dart"
  ]
}
```

```
created employee: id: gs123, name: GS retired,
salary: 0.0, hoursPerWeek: 10.0
languages: [C#, dart]
```

Issues in Deserialization

```
{
  "id": "gs123",
  "name": "GS retired",
  "hoursPerWeek": 10,
  "salary": 0.0,
  "languages": [
    "C#",
    "dart"
  ]
}
```

```
24    }
25    var result = Employee(json["id"]);
26    result.name = json['name'];
27    result.hoursPerWeek = json['hoursPerWeek'];
```

Exception has occurred. ×
_TypeError (type 'int' is not a subtype of type 'double')

avoid exception:

```
if (json['hoursPerWeek'] is num) {
  result.hoursPerWeek = json['hoursPerWeek'].toDouble();
}
```

This also helps when in the file there is no “hoursPerWeek” entry at all, e.g. because the file was written from an older version that did not manage “hoursPerWeek”.

Code from Gemini (https://gemini.google.com/share/0df6f069b448)

Please write toJson and fromJson for the following class

```
class Employee {
 Employee(this.id);
 String id;
 String name = "unknown";
 double hoursPerWeek = 40;
 double salary = 1234;
 List<String> languages = [];
}
```

Writing `toJson` and `fromJson` methods in Dart is a standard way to handle serialization. This allows you to easily convert your objects to a Map (which can then be turned into a JSON string) and back again.

Here is the updated `Employee` class with the necessary logic:

```dart
class Employee {
  String id;
  String name;
  double hoursPerWeek;
  double salary;
  List<String> languages;

  Employee(
    this.id, {
    this.name = "unknown",
    this.hoursPerWeek = 40,
    this.salary = 1234,
    this.languages = const [],
  });

  // Converts a Map (JSON) into an Employee instance
  Employee.fromJson(Map<String, dynamic> json)
      : id = json['id'],
        name = json['name'] ?? "unknown",
        hoursPerWeek = (json['hoursPerWeek'] as num?)?.toDouble() ?? 40.0,
        salary = (json['salary'] as num?)?.toDouble() ?? 1234.0,
        languages = List<String>.from(json['languages'] ?? []);

  // Converts an Employee instance into a Map (JSON)
  Map<String, dynamic> toJson() {
    return {
      'id': id,
      'name': name,
      'hoursPerWeek': hoursPerWeek,
      'salary': salary,
      'languages': languages,
    };
  }
}
```

Details on Gemini's fromJson

```
// Converts a Map (JSON) into an Employee instance
Employee.fromJson(Map<String, dynamic> json)
    : id = json['id'],
      name = json['name'] ?? "unknown",
      hoursPerWeek = (json['hoursPerWeek'] as num?)?.toDouble() ?? 40.0,
      salary = (json['salary'] as num?)?.toDouble() ?? 1234.0,
      languages = List<String>.from(json['languages'] ?? []);
```

Key Implementation Details

- **Type Safety:** For the `double` fields, I used `(json['field'] as num).toDouble()`. This is a best practice in Dart because JSON decoders often treat numbers without decimals as `int`, which can cause a type mismatch error if you try to cast them directly to `double`.
- **Null Safety:** I included null coalescing operators (`??`) to ensure that if a key is missing from the JSON, your default values remain intact.
- **List Handling:** `List<String>.from()` ensures that the dynamic list coming from JSON is correctly cast to a string list.

```
// Map -> Object
factory Employee.fromJson(Map<String, dynamic> json) {
  if (json["id"] == null) {
    throw Exception("no field 'id' found in $json");
  }
  var result = Employee(json["id"]);
  result.name = json['name'];

  if (json['hoursPerWeek'] is num) {
    result.hoursPerWeek = json['hoursPerWeek'].toDouble();
  }
  result.salary = json['salary'];
  //result.languages = json['languages'];
  //print(json['languages'].runtimeType); // List<dynamic>
  result.languages = List<String>.from(json['languages']);
  return result;
}
```

More robust version of “our fromJson”

```
// Map -> Object
factory Employee.fromJson(Map<String, dynamic> json) {
  if (json["id"] == null) {
    throw Exception("no field 'id' found in $json");
  }
  var result = Employee(json["id"]);
  result.name = json['name'];
  result.hoursPerWeek = json['hoursPerWeek'];
  result.salary = json['salary'];
  print(json['languages'].runtimeType); // List<dynamic>
  result.languages = List<String>.from(json['languages']);
  return result;
}
```

```
{
  "id": "gs123",
  "nameX": "GS retired",
  "hoursPerWeek": "a",
  "salary": 0.0,
  "languages": [
    1,
    "dart"
  ]
}
```

```
// Map -> Object
factory Employee.fromJson(Map<String, dynamic> json) {
  if (!(json["id"] is String)) {
    throw Exception("no field 'id' found in $json");
  }
  var result = Employee(json["id"]);
  if ((json["name"] is String)) {
    result.name = json['name'];
  }
  if ((json["hoursPerWeek"] is num)) {
    result.hoursPerWeek = json['hoursPerWeek'].toDouble();
  }
  if ((json["salary"] is num)) {
    result.salary = json['salary'].toDouble();
  }
  print(json['languages'].runtimeType); // List<dynamic>
  if ((json["languages"] is List<String>)) {
    result.languages = List<String>.from(json['languages']);
  }
  return result;
}
```

Bad formatted json file

```
{
  "id": "gs123",
  "name": "GS retired",
  "hoursPerWeek": x,
  "salary": 0.0,
  "languages": [
    "C#",
    "dart"
  ]
}
```

```
28    final Map<String, dynamic> readMap = jsonDecode(readFromFile); readFromFi

Exception has occurred. ×
FormatException (FormatException: Unexpected character (at line 4, character 19)
  "hoursPerWeek": x,
                  ^
)
```

```dart
String readFromFile = await file.readAsString();
print(readFromFile);

Map<String, dynamic> readMap = {};
try {
  readMap = jsonDecode(readFromFile);
} on Exception catch (e) {
  print("exception caught: $e");
  return;
}
final created = Employee.fromJson(readMap);
```

In this lesson you should have learned to

- use the type “dynamic” in Dart
- describe the difference between a List and a Map (Dictionary) in Dart
- serialize members of a class into a json file and deserialize them from there
- use the keywords async and await in Dart